

Prototype design

idea - "to reuse an existing design in place of designing from scratch"

- existing design is prototype
- we copy it and make changes accordingly.
- to make cloning convenient, we use prototype factory ✓

we use the concept of deepcopy here to make clone object.

Class Address:

```
def __init__(self, street, city, country):  
    self.city = city  
    self.country = country  
    self.street = street  
def __str__(self):  
    return f'{self.street}, {self.city}, {self.country}'
```

class Person:

```
def __init__(self, name, address):  
    self.name = name  
    self.address = address
```

```
def __str__(self):  
    return f'{self.name} lives at {self.address}'
```

```
address = Address('123 st', 'mumbai', 'india')
```

```
person1 = Person('lit', address)
```

```
person2 = copy.deepcopy(person1)
```

```
person2.name = 'Aru'
```

```
person2.address.street = 'Andheri'
```

deepcopy creates a completely independent copy of an object, recursively copying all of the objects it contains.

Now the problem here is that we should follow the proper design patterns to initiate the cloning, to do so, we can create a factory class.

class EmployeeFactory:

```
tech-office-emp = Person("", Address("123", "ABC", "india"))  
marketing-office-emp = Person("", Address("456", "EFG", "india"))
```

@staticmethod

```
def _new_employee(proto, name):  
    result = copy.deepcopy(proto)  
    result.name = name  
    return result
```

@staticmethod

```
def new_tech_emp(name):  
    return EmployeeFactory._new_employee(  
        EmployeeFactory.tech-office-emp, name)
```

@ static method

def new-marketing_emp(name):

return EmployeeFactory.new_employee(
EmployeeFactory.marketing_office_emp, name)

lit = EmployeeFactory.new_tech_employee("lit")